Brightloop — Final Engineering Evaluation

Final self-assessment: 93 / 100 Outcome: polished, evidence-backed hackathon project; the requested >90 target is met under the original rubric.

Date: 31 October 2026. Baseline: `f228f3b7a3029d9563e9be8c4dcdf63a6fe39eb7` (76/100). The same attached four-page Subscription Management System problem statement and the same weighted rubric were used throughout. This is an engineering self-assessment of the implemented changes, **not external jury certification or a production-readiness percentage**.

1. Iterative improvement record

Stage	Work	Result	Score
Baseline	Independently cloned/evaluated original submission	Working broad prototype, financial/date/delivery gaps	76
Iteration 1	Immutable pricing, accrued billing, old-record visibility, finance reconciliation, money validation and lifecycle	28 SQLite tests + build passed	84
Iteration 2	SMTP outbox/reset links, contract/add-on amendments, scoped server search/pickers/aggregates, session revocation	33 backend tests + 10 API browser checks passed	89
Iteration 3	PostgreSQL parity, concurrency/rollback, migrations, quality/security gates, realistic browser/API acceptance, visual preservation and operations	Final verification below	**93**

Each iteration documents what changed, why, test failures/corrections, evidence, scoring impact and remaining recommendations:

- `docs/iterations/01-financial-correctness.md`
- `docs/iterations/02-complete-workflows-and-delivery.md`
- `docs/iterations/03-hardening-and-final-acceptance.md`

The loop stopped after the evidence-supported score exceeded 90; it was not repeated merely to relabel unchanged software with a larger number.

2. Final scorecard

Category	Weight	Baseline	Final	Evidence / residual qualification
Accounts, authentication and access	12	10	**12**	Password/unique-email/RBAC tests; clickable reset links with real SMTP adapter; access revocation; refresh rotation.
Products, variants and plans	13	12	**13**	Catalog, variant deltas, four frequencies, quantities/availability/action flags; searchable selection and add-on management.
Quotations and lifecycle	13	9	**12**	Immutable offers, transactional conversion, complete state route, contract amendments and revision conflicts. Large workflow module/legacy re-quote migration constraint remain.
Billing, invoices and rules	20	14	**18**	Preserved accrual, paused intervals, anchor recovery, immutable invoice approval, atomic limits, concurrent billing, PDFs and SMTP attachments. Historical rule reconstruction and richer accounting policies remain.
Payments and receivables	9	8	**9**	Precision/overpayment/partial state, debt reconciliation, payment idempotency and PostgreSQL numeric handling.
Customer self-service	6	5	**6**	Ownership enforcement for lists/details/PDF/actions/amendments; quantities/add-ons; no customer contract override.
Finance and performance	7	5	**6**	Source-ledger cash analytics, all-history debt, SQL aging, server aggregates and realistic local page/read acceptance. WAN/slow-device/high-concurrency production guarantees remain unproven.
Architecture / maintainability	5	3	**3**	Dedicated migration/mail/editor/picker modules and explicit policies, but large main app/route files and three documented dynamic-type boundaries remain. No inflated architecture credit.
Testing / quality gates	6	4	**6**	SQLite + PostgreSQL, real SMTP socket, genuine upgrade, overlap/rollback, browser, coverage, audit and CI definition.
Security / operations	5	3	**4**	Clean current audit, security headers, immediate session revocation, durable retries, backups and deployment packaging. localStorage tokens, independent security review and deployment rehearsal remain.
UX / documentation / reproducibility	4	3	**4**	Same CSS and measured desktop/mobile layout; existing-pattern controls; runnable source, detailed iterations, policies, evidence and honest limits.
Total	**100**	**76**	**93**	**Functional: 76/80. Engineering: 17/20.**

Deductions are applied primarily to their owning category rather than multiplying the same defect across unrelated categories. A high hackathon score does not remove launch blockers for real-world financial/compliance requirements.

3. Final verification results

Verification	Observed result
Lint	**Pass**, frontend/backend/scripts; no errors/warnings in final gate
Type-check + production build	**Pass**
SQLite integration/regression/upgrade	**39/39 pass**
PostgreSQL 17 core/hardening	**38/38 pass** (15 core + 23 hardening); original-schema upgrade fixture is SQLite-only
Instrumented non-test backend coverage	**89.04% lines, 78.17% branches, 83.24% functions**; thresholds passed
Demo browser suite	**10/10 pass**
Actual API-connected browser suite	**10/10 pass**
Real SMTP controlled receiver	Clickable reset link and PDF attachment received; retry/concurrent claim tests pass
Overlapping billing	Two workers against 500 due subscriptions: **500 invoices, zero duplicate periods/failures**, tested on both DB drivers
Financial failure recovery	Invalid pricing rolls back cursor/invoice; failed debt is not auto-closed away; corrected import retries successfully
Original-schema upgrade	Existing user/subscription/quote retained, defaults applied, versions repeat safely
Dependency audit	**0 known findings** at recorded run, including development dependencies
Production native-process smoke	Static/API/auth/security headers/no external recording hooks and consistent SQLite backup/integrity check pass
Visual regression	Main CSS hashes unchanged; eight dashboard regions match at 1440px and 390px; no horizontal overflow

Fresh outputs are under `docs/evidence/`. Final `npm run check` output is authoritative over earlier intermediate failure logs. Coverage measures loaded backend implementation modules, not all frontend/deployment code. The CI workflow is included and its component commands were run locally; no hosted CI success or Docker-engine build is claimed.

4. Performance evaluation — stronger than the baseline

The original benchmark expanded only subscriptions and retained a small demonstration finance ledger. The new acceptance script builds an API-connected frontend, seeds **10,000 subscriptions, 60,000 invoices and 30,000 payments**, serves actual HTTP/API responses, opens the application in Chromium and runs overlapping billing against a 500-due subset.

Measured scope

• Eight real HTTP endpoints: 50 requests each, concurrency five; response bodies fully read/parsed.

• Five overview navigations/reloads, including initial sign-in.

• Five visits each to Subscriptions, Invoices, Payments, Customers, Products & plans, Quotations, Taxes & discounts, Reports, Audit log and Settings.

• SPA readiness explicitly tracks in-flight fetches, a short network-stability period and two rendering frames; it does not just wait for the title to appear.

• Exactly 500 new invoices from the due subset with two overlapping billing jobs.

The recorded final ledger run passed the two-second checks. The included JSON contains exact API percentiles, per-page samples and billing counts/times. An earlier recorded run showed overview loads around **0.5–1.0 seconds**, page transition sampled maxima below **0.4 seconds**, and API p95 values below **0.4 seconds** after SQL aging aggregation.

Limits

This is a local Node 24 / in-memory SQLite / headless Chromium test. Five browser samples make the nearest-rank p95 effectively the sampled maximum; they are not statistically strong production-SLA evidence. PostgreSQL correctness/concurrency is tested, but its mixed-ledger WAN/browser latency is not. Slow phones, internet/provider delays and hundreds of simultaneous users remain deployment acceptance work. Accordingly, finance/performance is 6/7, not a perfect guarantee.

5. Closure of the original findings

Original finding	Implemented correction	Acceptance evidence
Pause/resume skips old billing	Preserve cursor; record completed pause intervals; skip only eligible paused starts	Regression + overlap + recovery tests, both drivers
Quote reprices on conversion	Snapshot server-calculated offer lines and copy accepted prices	Change plan 100 → 200 after quote; converted price remains 100
Invoice confirmation changes discounts/taxes	Approve stored draft snapshot; atomic usage consumption; conflict rather than silent revision	Historical offer/rule-change and last-allowance race tests
Old active records disappear	All-history operational default; explicit optional filters	Old active product/subscription + >100 customer browser search
Drafts count as collectible debt	Only Confirmed open balances are receivables/overdue/reminder targets	Draft exclusion/old-debt reconciliation tests
Aging omits old debt	All-history as-of SQL aggregation; visible explicit bounds	Two-year-old debt reconciles to outstanding
Missing Draft → Quotation	Added server action and existing-style drawer control	Full lifecycle sequence test
Sub-cent payment becomes zero	Precision validation and finite decimal normalization	0.001 rejected; NaN rollback on PostgreSQL
No real reset/invoice email	SMTP transport, reset links, MIME PDF attachments, durable retries/status/leases	Controlled SMTP receipt/failure/retry tests
Cached subset used as global data	Server search, searchable customer/plan selection, upcoming renewals and aggregate-backed overdue values	Old/unloaded search and >100 selector browser checks
No ongoing line/add-on workflow	Staff/customer prospective amendments, safe catalog, revisions, server prices	Add/remove quantity/add-on, timing and ownership tests
Billing anchor drifts	Original-day anchored period advancement	Month-end and leap-year tests
Historical materializations incomplete	Reports aggregate source payments; current stock summaries refresh	Historical import/source-payment regression
Orphaned tables after repeated navigation	Distinct stable sibling key namespaces; repeated navigation/console-error assertions	High-volume browser loop and final screenshot
Lint/dependency gaps	Fixed meaningful violations, backend lint, patched compatible deps, removed unused router, documented override	Clean check and zero-audit JSON

Original finding	Implemented correction	Acceptance evidence
PostgreSQL only claimed	Isolated actual PostgreSQL runner, invoice/discount/payment/mail concurrency	38 executed cases, including overlapping workers
No upgrade path	Three additive versioned migrations + original-schema fixture	Record retention / repeatability test
Limited operations/deployment	Production same-origin serving, security headers, Dockerfile, backup utility, recovery docs	Native smoke + backup/integrity check; container build explicitly unverified

These fixes address both the explicit PS gaps and the professional-engineering risks found during the first evaluation. Policy choices are explicit rather than treated as universal tax/accounting law.

6. PS alignment after remediation

Signup and access (PS p2)

Implemented: one normalized email per account; >8-character/uppercase/lowercase/special password rule; customer signup; expiring single-use reset link; admin/staff/customer separation; admin-only staff creation; server-owned customer scope. SMTP is an actual supported delivery mechanism, while local preview remains honestly labeled. The deployment must supply working provider credentials.

Product and plan setup (PS p3)

Implemented: product name/type/sales/cost/notes, variant deltas (including Odoo +560), four frequencies, minimum plan quantity, availability and plan-specific close/pause/renew policies. Catalog reads/writes remain role-restricted; customers receive only a safe add-on selection surface, not internal costs.

Quote and subscription (PS p3)

Implemented: plan/product templates with validity; immutable offer conversion; one-time/expired quote controls; numbered subscription, customer/plan/start/expiry/terms and lines; full stated lifecycle; team status visibility; contract and prospective line edits. Subscription detail displays price/amount and a clearly labeled current tax estimate, while invoices contain authoritative snapshots. Legacy unsnapshotted quotes must be explicitly re-created rather than pretending their old agreed price is knowable.

Billing day (PS p3)

Implemented: due scans/backfill, transactional per-period invoice generation, customer/product/tax/total snapshots, invoice confirmation/cancellation/send/PDF print/download and paid-state transitions. Database period uniqueness and tested overlapping workers protect duplicate billing. Recorded pause/close policies preserve earned service. Failures leave a recoverable cursor and are logged.

Discounts and payments (PS p4)

Implemented: admin-only fixed/percent rules, minimum amount/quantity, dates/usage caps, product/subscription scope and automatic invoice application. Confirmation never silently changes terms, and caps are transactional. Payment method/amount/date/note link to invoices; partial and exact balance, rejection and idempotent retry behavior are verified.

Finance and speed (PS p4)

Implemented: active counts, collected receipts, outstanding/overdue, filters, summaries, aging and actual-payment charts; all-history operational visibility and server-based global queries. Local mixed-ledger page acceptance passes. The universal phrase “every page under two seconds” still requires target-device/network/concurrency validation in the deployment; it is not converted into an unconditional claim.

7. UI/UX preservation

Both original stylesheets are unchanged, with hashes recorded in `iteration-3-visual.json`. The original source commit was built for comparison. At 1440×1060 and 390×844, measured dashboard geometry and computed typography/color/backgrounds match across eight key regions, with no horizontal overflow.

New controls—Prepare quotation, Edit subscription, optional term expiry and searchable pickers—reuse the existing field/button/modal patterns. No branding, palette, navigation, card-grid or typography redesign was introduced. Dynamic numbers and explanatory text can differ because financial calculations were corrected. This evidence does not assert every pixel of every new modal is identical to a screen that previously had no such workflow.

Original external recording/CDN/page-view/editor code was removed from the shipped HTML; it had no required visual function and caused unnecessary external requests.

8. Why the final score is 93, not 100

Remaining seven points are intentionally withheld:

• **Quotes/lifecycle: −1.** The workflows are substantially complete, but legacy unsnapshotted offers require explicit re-quote and more complex commercial approval/versioning is not built.

• **Billing/rules: −2.** No historical tax-rule version ledger or jurisdiction-certified accounting/proration/credit-note model. Confirmation snapshots are stable; arbitrary historical tax reconstruction is not promised.

• **Finance/performance: −1.** Strong local evidence, not a universal production/browser/network SLA.

• **Architecture: −2.** Large main frontend/route files and three narrow dynamic legacy boundaries remain. Formatting/new modules do not erase this debt.

• **Security/operations: −1.** localStorage tokens and deployment-specific review/restore/delivery/container validation remain.

Payment gateways, microservices, Kubernetes, multi-currency, metered usage, automated bank feeds and MRR/ARR are not silently introduced as mandatory requirements absent from the attached PDF. They may be valuable later, but were not used to manufacture deductions or scope.

9. Hackathon demonstration plan

1. Run from README on Node 24; choose the actual API-connected mode.

2. Register a customer; demonstrate weak-password/duplicate-email rejection and staff access restrictions.

3. Add product/variant/plan/template; create quote, alter catalog price and show preserved conversion terms.

4. Demonstrate Draft → Quotation → Confirmed → Active and customer-owned self-service.

5. Open existing-design contract editor; schedule quantity/add-on changes, show effective date and ownership restrictions.

6. Run billing twice; show unique periods, correct immutable invoice/PDF and notification delivery status.

7. Record partial/final payment; demonstrate safe repeated request keys and paid-state/report reconciliation.

8. Show old active records/old debt, server search and the realistic benchmark JSON.

9. Show clickable reset email / invoice attachment through configured SMTP or clearly labeled local preview.

10. Present iteration docs, final tests and honest limitations. Do not describe localhost measurements or a self-score as external certification.

10. Reproduction / deliverables

Core commands:

```
npm ci
npm run check
npm run test:coverage
npm audit
npx playwright install --with-deps chromium
npm run test:ui
npm run test:local-ui
npm run test:acceptance
npm run test:production
TEST_DATABASE_URL=postgresql://user:password@localhost:5432/test_db npm run test:postgres
```

Optional original visual comparison requires a separately built baseline checkout at `artifacts/baseline-source` (or `BASELINE_DIR`), then `npm run test:visual`. The repository's original commit and CSS hashes are recorded. Do not run load tests against real customer databases; supplied acceptance tests use isolated in-memory data, PostgreSQL tests use randomized dedicated schemas.

The source download includes implementation, lockfile, schema upgrades, tests, workflow/container configuration, iteration docs, final evaluation and fresh evidence. No `.env`, real database, node_modules, runtime logs or SMTP credentials are included. README and `docs/OPERATIONS_AND_POLICIES.md` define setup and policies. The original source ZIP was regenerated to avoid delivering a stale prototype archive.

Final jury-style statement: “Brightloop now demonstrates a coherent full-stack subscription business workflow with substantially improved financial correctness, real delivery capability, customer self-service, cross-database tests and reproducible quality evidence. I assess it at **93/100 for a polished hackathon submission**, while retaining explicit deductions and deployment caveats rather than claiming production perfection.”